Reunite: Beginner's Guide (Python + React for Angular developers)

Oct 25, 2026 · @Someone

Your two questions, answered first

The database does not store the image itself. The photo is saved as a normal `.jpg` file on disk (or in a private Supabase bucket), and the database row only stores the file's name plus some numbers about the face.

1. How images are stored

Think of it like Angular's `assets/` folder versus a JSON API: the heavy file lives in one place, and a small record points to it.

Where	What is kept	Example
File storage (`backend/data/uploads/` locally, or a private Supabase bucket in production)	The photo bytes, as JPEG	`sighting/9f3c.../original.jpg`
Database table `sightings` or `missing_persons`	The file's name (`original_key`, `restored_key`)	`original_key = "sighting/9f3c.../original.jpg"`
Same database row	The face "fingerprint" (`embedding`): a list of 512 decimal numbers	`[0.012, -0.093, 0.221, ...]`
Same database row	Face count, detection score, status, and for sightings the GPS location	`face_count = 1`, `lat = 17.385`

Why this split: a database is built for small searchable values, not for megabytes of pixels. The 512 numbers are what get compared when looking for matches; the JPEG is only shown to humans.

The photo is never reachable by a public URL. The browser asks `GET /api/files/sighting/<id>/original` with your login token, the server checks whether you are allowed to see it, and only then sends the bytes. The React component `SecureImage.jsx` receives those bytes and displays them.

Code to read: `backend/app/services/storage.py` (saving and loading files) and the `Sighting` class in `backend/app/models.py` (the columns).

2. Is any AI transformation applied to the uploaded image?

Yes, but only in specific, mostly optional steps, and the original is always kept. Here is exactly what happens to a photo, in order:

Step	Where in code	Is it AI?	Changes the pixels?
Check it is a real image, fix rotation, shrink to at most 2000 px, re-save as JPEG (this also removes hidden metadata such as GPS)	`services/images.py`, `sanitize_image`	No, ordinary image processing	Yes, slightly (resize + recompress). This copy is what we call the "original"
Read GPS from the photo's hidden metadata (only if the user gave no location)	`services/images.py`, `read_exif_gps`	No	No
Find the face and turn it into 512 numbers	`services/faces.py` (InsightFace model)	**Yes, a face-recognition model**	**No**, it only reads the photo

Step	Where in code	Is it AI?	Changes the pixels?
Cleanup, default mode `basic`: enlarge small photos, stretch contrast, light sharpen	`services/images.py`, `basic_enhance`	No, classic filters	Yes, but it cannot invent facial detail
Cleanup, optional modes `gfpgan` or `hf_space` (CodeFormer)	`services/restore.py`	**Yes, AI face restoration**	Yes, and it can subtly change a face. **Off by default**
Safety check: compare the face before and after cleanup; throw the cleaned copy away if it no longer looks like the person	`services/pipeline.py`	Uses the face model	No
Compare the 512 numbers against stored records and list the closest	`services/matching.py`	Maths (cosine similarity), not generation	No

So, in the default setup, **no AI ever generates or redraws a face**. The only AI is the face-recognition model, which looks at the photo and outputs numbers. The cleaned copy is made by plain filters, and is saved as a separate file (`restored.jpg`) next to the original.

Three things to remember:

- **Turning AI cleanup on is a choice you make** in `backend/.env` with `RESTORE_BACKEND=gfpgan` or `hf_space`. The Hugging Face option sends the photo to a third-party server, so the privacy page must say so first.
- **If you run with `FACE_BACKEND=mock`** (the quick-start demo), there is no AI at all: it compares a blurry brightness pattern, which is only good enough to test the screens.
- **The AI never makes the final decision.** It only suggests pairs. A human moderator compares the original photos and confirms or rejects every match.

One honest detail: the very first file we save is already the resized, recompressed copy. The raw bytes the user uploaded are not kept.

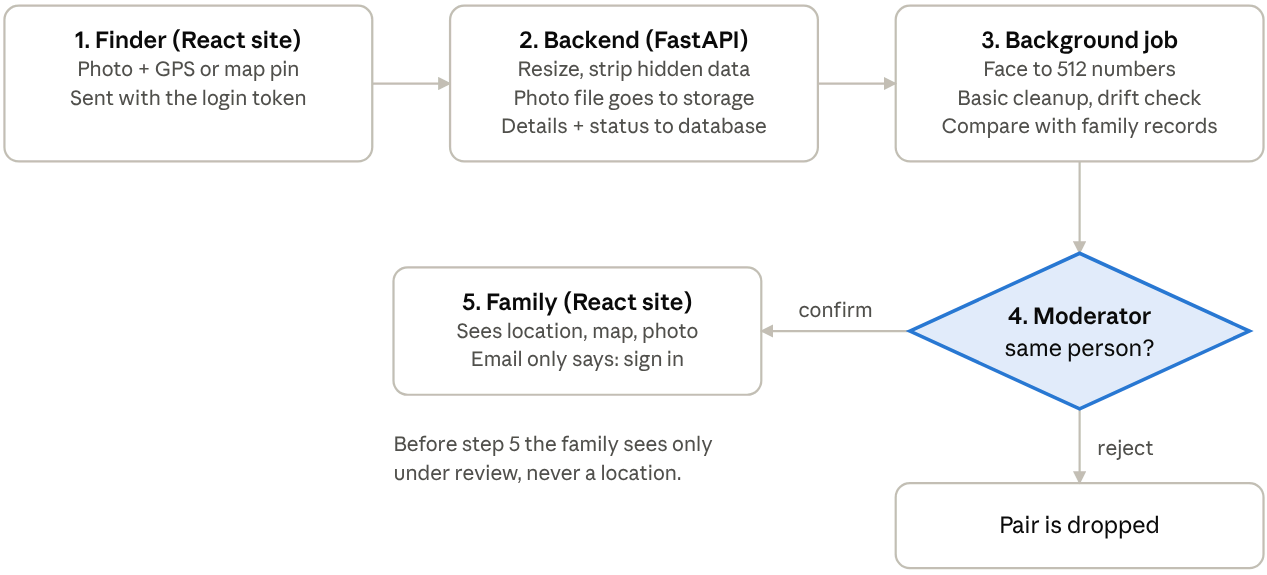
The big picture

Reunite is a matching service in which software suggests pairs and a human moderator decides. Families register someone they are looking for. Members of the public upload a photo of someone they saw, with a location. The software compares faces and flags look-alikes, and nobody learns anything until a trained person confirms.

The pieces

- **React site** (`frontend/`): what people see in the browser. It runs on your computer or on a static host.
- **FastAPI server** (`backend/`): the Python program that checks logins, saves photos, runs the AI, and answers the React site.
- **Database and photo storage**: the database holds the facts and the face numbers; a separate folder (or private bucket) holds the photo files.
- **Humans**: moderators who approve records and confirm matches. They are the real safety layer.

Software suggests, a moderator decides before anyone sees a location



upload to confirmed match · 5 steps, 1 human decision

A photo travels left to right through steps 1 to 3 on its own. Step 4 is the only place a decision is made, and it is made by a person looking at the original photos, not at the score. Only a confirmed pair reaches step 5.

Who can see what

Person	Can see	Cannot see
Public visitor	The home page and privacy page	Any photo, record, or location
Finder (uploaded a sighting)	Their own uploads	Family details, or whether any match happened
Family	Their own records. After a moderator confirms: the exact location, map and sighting photo	Locations of unconfirmed matches, other families' records
Moderator	All photos, locations, contact phone numbers, and the audit log	Nothing is hidden, so every action they take is logged

Python in 15 minutes, for someone who knows TypeScript

Python reads like pseudo-code: no braces, no semicolons, and **indentation is the syntax**. If you can read TypeScript, you can read 90% of this project after seeing the table below.

TypeScript to Python, side by side

Idea	TypeScript / Angular	Python (this project)
Variable	`const name = "Asha";`	`name = "Asha"`
Typed variable	`let age: number | null = null;`	`age: int | None = None`
Function	`function add(a: number, b: number): number { return a + b; }`	`def add(a: int, b: int) -> int:` then an indented `return a + b`
Block	`{ ... }`	A colon `:` then indented lines (4 spaces)

Idea	TypeScript / Angular	Python (this project)
null / undefined	`null`, `undefined`	`None`
true / false	`true`, `false`	`True`, `False` (capital first letter)
Comparison	`===`, `&&`, `||`, `!`	`==`, `and`, `or`, `not`
String template	`` `Hi ${name}` ``	`f"Hi {name}"`
Array / object	`[1,2]`, `{a: 1}`	`[1, 2]` (list), `{"a": 1}` (dict)
Loop	`for (const x of items)`	`for x in items:`
Transform a list	`items.map(x => x * 2)`	`[x * 2 for x in items]` (list comprehension)
Import	`import { Foo } from './foo'`	`from .foo import Foo`
Class	`class A { constructor(public x: number) {} }`	`class A:` with `def __init__(self, x):`
Package manager	`npm install` and `package.json`	`pip install` and `requirements.txt`
Comment	`// note`	`# note`

Two more differences that trip people up:

- **`self` is explicit.** Inside a class, Python methods list the object as their first parameter (`def embed(self, data)`), where TypeScript uses an invisible `this`.
- **Type hints are optional hints, not rules.** Python will run `age: int = "hello"` without complaint. This project uses hints so your editor can help, and FastAPI also uses them to validate incoming requests.

Setting up Python (like `node_modules`, but called a virtual environment)

```
cd backend
python -m venv .venv            # creates a private folder of packages, like node_modules
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt  # like npm install
```

The `(.venv)` prefix in your terminal tells you it is active. If you forget to activate it, you get `ModuleNotFoundError`, which is the Python version of "module not found".

Five Python ideas you will meet in this project

1. Decorators are the `@something` lines above a function. They are the same idea as Angular's `@Component` or `@Injectable`: they attach behaviour to what follows.

```
@router.get("/mine")          # "when someone sends GET /mine, run this
function"
def my_records(...):
    ...
```

2. Dependency injection exists here too. Angular injects services through the constructor. FastAPI injects them through default values with `Depends(...)`:

```
def my_records(user: User = Depends(get_current_user), db: Session =
Depends(get_db)):
```

Read it as: "before running this function, call `get_current_user` and `get_db`, and hand me their results." That is why every protected endpoint is only one line of security.

3. Classes as data shapes. A class in `models.py` describes a database table (like a TypeORM entity). A class in a router file that extends `BaseModel` describes the JSON a request must contain (like a TypeScript interface, but checked at runtime).

4. `with` blocks open something and always close it, even after an error:

```
with SessionLocal() as db:   # opens a database session, closes it at the end
of the block
    ...
```

5. `try` / `except` / `raise` are `try` / `catch` / `throw`. In this project, `raise HTTPException(404, "Not found")` stops the function and sends a 404 reply to the browser.

Sync vs async

Angular code is full of `async` / `await` and `Observable`. In this backend nearly every function is a plain `def`, which FastAPI runs on a worker thread so one slow request does not block the others. You can read it top to bottom like ordinary code. The one exception is `async def lifespan(...)` in `main.py`, which runs once at startup.

Files and folders are modules

A folder with an `__init__.py` is a Python package. `from .services import jobs` means "from the `services` folder next to me, load `jobs.py`". The leading dot means "relative to this folder", like `./` in TypeScript.

How Python finds the APIs (Spring Boot comparison)

FastAPI does not scan for controllers. Every router is registered by hand in `main.py`. Spring Boot's `@SpringBootApplication` searches the classpath for `@RestController` classes. Python has no such step: if a router file is never imported and included, its URLs simply do not exist.

The mapping

Spring Boot	FastAPI in this project
`mvn spring-boot:run`, embedded Tomcat	`uvicorn app.main:app`: the server. It loads the file `app/main.py` and uses the variable `app`
`@SpringBootApplication` main class	`app/main.py`, where `app = FastAPI(...)` is created
Component scan	**None.** `app.include_router(...)` is called once per router file
`@RestController` + `@RequestMapping("/api/missing")`	`router = APIRouter(prefix="/api/missing")` in `routers/missing.py`
`@GetMapping("/mine")`	`@router.get("/mine")` above a function
`@RequestBody MyDto dto`	A parameter whose type is a `BaseModel` class
`@RequestParam`, `@RequestPart MultipartFile`	`Form(...)` and `File(...)` parameters
`@PathVariable`	`{missing_id}` in the path, plus a `missing_id: str` parameter
`@Autowired` service	`Depends(...)` default value

Spring Boot	FastAPI in this project
Spring Security filter chain	`Depends(get_current_user)` and `Depends(require_moderator)`
`@ExceptionHandler`, `ResponseStatusException`	`raise HTTPException(404, "...")`
`OncePerRequestFilter`	`@app.middleware("http")` function `guard` in `main.py`
`@PostConstruct`, `ApplicationRunner`	`lifespan(...)` in `main.py`, which runs once at startup
`@Async`	`BackgroundTasks.add_task(...)`
`application.properties`	`config.py` reading `.env`
`@Entity`, Spring Data JPA	`models.py` with SQLAlchemy, queried by `db.query(...)`
`spring.jpa.hibernate.ddl-auto=update`	`Base.metadata.create_all(engine)` in `db.py`. It creates missing tables but never alters existing ones
springdoc / Swagger UI	Built in: open `http://localhost:8000/docs`

What happens when you start the server

1. `uvicorn` imports `app/main.py`. Python runs a file's top-level lines in order, as soon as it is imported.
2. The line `from .routers import admin, auth, files, ...` imports each router file. While each one loads, its `@router.get(...)` and `@router.post(...)` lines run and **add that function to the router's own list of routes**. Nothing is public yet.
3. `app = FastAPI(...)` is created, then CORS and the `guard` middleware are added.
4. This is the line that plays the role of component scanning, and it is a plain loop:

```
for r in (auth, missing, sightings, matches, files, reports, admin):
    app.include_router(r.router)
```

5. The server starts listening. The `lifespan` function runs once: in production it refuses weak secrets, then `init_db()` creates tables, and `_fail_interrupted_jobs()` marks photos left in "processing" by a previous crash as failed.

6. For each request, FastAPI finds the matching route, runs every `Depends(...)` first (login check, database session, rate limit), validates the inputs from the type hints, runs your function, and turns the returned dict or `BaseModel` into JSON.

The trap to remember: if you add `routers/notes.py` and forget to import it and add it to that loop in `main.py`, you get `404 Not Found` and no error message at startup. Spring would have found it on its own.

Check it yourself

Three ways to confirm which URLs are live:

1. **Browser:** open `http://localhost:8000/docs`. It lists every registered endpoint, grouped by router, and you can call them from the page. (It is switched off when `ENV=prod`.)
2. **Terminal**, from the `backend` folder with the virtual environment active:

```
python -c "from app.main import app; [print(m.upper(), p) for p, ops in
sorted(app.openapi()['paths'].items()) for m in ops]"
```

3. **Tests:** `python -m pytest -q` calls the real endpoints through a test client.

This is the output of that command on the delivered code, 27 endpoints. Yours should match:

Router file	Endpoints
`auth.py`	`POST /api/auth/request-otp`, `POST /api/auth/verify-otp`, `GET /api/auth/me`, `DELETE /api/auth/me`
`missing.py`	`POST /api/missing`, `GET /api/missing/mine`, `GET /api/missing/{missing_id}`, `POST /api/missing/{missing_id}/close`, `DELETE /api/missing/{missing_id}`
`sightings.py`	`POST /api/sightings`, `GET /api/sightings/mine`, `GET /api/sightings/{sighting_id}`, `DELETE /api/sightings/{sighting_id}`
`matches.py`	`GET /api/matches/mine`, `POST /api/matches/{match_id}/feedback`

Router file	Endpoints
`files.py`	`GET /api/files/{kind}/{item_id}/{variant}`
`reports.py`	`POST /api/reports`
`admin.py`	`GET /api/admin/stats`, `GET /api/admin/missing`, `POST /api/admin/missing/{missing_id}/decision`, `GET /api/admin/matches`, `POST /api/admin/matches/{match_id}/decision`, `GET /api/admin/reports`, `POST /api/admin/reports/{report_id}/resolve`, `GET /api/admin/audit`
`main.py` itself	`GET /api/health`, `GET /api/config`

A practical way to verify the whole app: open `/docs`, click one endpoint, read its function in the matching router file, and follow any `Depends(...)` back to `security.py`. Do that for `GET /api/matches/mine` first, because it holds the most important privacy rule.

Backend tour: what every file does

The backend is a **FastAPI** app, a Python web framework that feels close to NestJS or an Express app with TypeScript. It exposes URLs (`/api/...`) that return JSON, and the React app calls them with `fetch`.

Angular vocabulary mapped to this backend

Angular / Node idea	In this project
`AppModule`, `main.ts`	`app/main.py` creates the app and plugs in every router
Controller / route	A function in `app/routers/*.py` with `@router.get(...)` or `@router.post(...)`
Service (`@Injectable`)	A module in `app/services/*.py` that does real work
Entity / TypeORM model	A class in `app/models.py`

Angular / Node idea	In this project
DTO / interface	`app/schemas.py` and the `BaseModel` classes inside routers
Guard / interceptor	`Depends(get_current_user)` and `Depends(require_moderator)` in `app/security.py`
`environment.ts`	`app/config.py`, filled from the `.env` file
Karma / Jest tests	`pytest` in `backend/tests/`

Top level of `backend/app`

File	What it does
`main.py`	Starts the app, allows the React site to call it (CORS), adds security headers, refuses to start in production with a weak secret, and registers all routers.
`config.py`	Every setting in one class. Values come from environment variables or `.env`. Change behaviour here without touching code (for example `FACE_BACKEND`, `RESTORE_BACKEND`).
`db.py`	Connects to the database. Uses SQLite (one local file) by default, or Postgres if `DATABASE_URL` says so, and creates the tables on startup.
`models.py`	The tables: `User`, `OtpCode`, `MissingPerson`, `Sighting`, `Match`, `Report`, `AuditLog`. Start here to understand the data.
`schemas.py`	The exact fields we are willing to send back to the browser. Sensitive columns (like the embedding and exact location) are left out on purpose.
`security.py`	One-time-code creation and hashing, login tokens (JWT), "who is calling?", "is this a moderator?", and the rate limiter.

`app/routers/` : the URLs

File	URLs it serves	Who may call
`auth.py`	Request a code, verify it, `GET /me`, delete my account	Anyone (to sign in), then signed-in users
`missing.py`	Register a missing person, list mine, close, delete	Signed-in family
`sightings.py`	Upload a sighting with photo and location, list mine, delete	Signed-in finders
`matches.py`	Show my matches (details only once confirmed), answer "reunited / not them"	Family who owns the record
`files.py`	Send a photo, only after checking permission	Owner, moderators, or the family of a confirmed match
`reports.py`	Abuse or takedown form	Anyone, no account needed
`admin.py`	Review queues, approve records, confirm or reject matches, takedowns, audit log	Moderators only

`app/services/` : the real work

File	What it does
`images.py`	Checks an upload is a valid image, shrinks and re-saves it, reads GPS from photo metadata, and runs the basic cleanup filters.
`faces.py`	Turns a face into 512 numbers. Three interchangeable engines: InsightFace, DeepFace, or a fake `mock` one for demos.
`restore.py`	Optional cleanup engines (GFPGAN, Hugging Face). Falls back to the basic filter if one fails.
`pipeline.py`	The order of operations on a photo: embed, clean, embed again, drop the cleaned copy if the face drifted.

File	What it does
`matching.py`	Finds the most similar faces. On Postgres this is a database query; on SQLite it is a loop in Python. Saves suggested pairs as `Match` rows.
`geo.py`	Turns latitude and longitude into a street address using OpenStreetMap, at most one request per second.
`storage.py`	Saves and loads photo files: local folder or Supabase bucket.
`jobs.py`	The background tasks that run after an upload so the user does not wait.
`notify.py`	Sends email (SMTP) and Telegram alerts. In development it just prints emails to the terminal.
`records.py`	Deletes a record together with its files and matches, and writes audit-log lines.

Outside `app/`

File	Purpose
`requirements.txt`	The lighter packages needed to run everything (like `dependencies` in `package.json`).
`requirements-ml.txt`	The heavy AI packages (InsightFace, ONNX Runtime, OpenCV). Install only when you want real face matching.
`requirements-optional.txt`	Extras: GFPGAN, Hugging Face client, iPhone HEIC support.
`.env.example`	A commented list of every setting. Copy it to `.env` and edit.
`tests/`	Automated tests. `test_flow.py` walks through the whole product.
`Dockerfile`	Recipe to package the backend into a container for deployment.

Which tables exist and how they connect

- A `User` owns many `MissingPerson` rows (families) and many `Sighting` rows (finders).
- A `Match` links exactly one `MissingPerson` to one `Sighting`, with a score and a status: `pending`, `confirmed`, or `rejected`.
- `OtpCode` holds the hashed sign-in codes, `Report` holds abuse reports, and `AuditLog` records what moderators did.

Follow one request: someone uploads a sighting

The fastest way to understand a codebase is to follow one real action through every file it touches. This one starts in the browser and ends with a moderator reviewing a possible match.

Part A: in the browser (React)

1. **`ReportSighting.jsx`**: the user picks a photo. The `onPhoto` function immediately asks the browser for GPS with `navigator.geolocation.getCurrentPosition`. If the user refuses, they can click the map (`Maps.jsx`) to drop a pin.
2. The user ticks the consent box and presses *Send report*. The `submit` function packs everything into a `FormData` (the standard browser object for file uploads).
3. **`api.js`**: `api.postForm("/api/sightings", form)` sends it with the login token in the `Authorization` header.

Part B: the request hits the backend (Python)

4. **`main.py`, middleware `guard`**: rejects anything bigger than about 11 MB before reading it, and adds security headers to the reply.
5. **`routers/sightings.py`**, **`create_sighting`**: FastAPI matches `POST /api/sightings` to this function. Before the function body runs, two dependencies fire:
   - `user_rate_limit("sighting-create", 10, 3600)` first calls `get_current_user` in `security.py`, which decodes the token and loads the user from the database, then allows at most 10 uploads per hour per user.
   - `get_db` opens a database session.
6. FastAPI validates the form fields from the type hints: `lat` must be between -90 and 90, and so on. A bad value gives an automatic 422 error with no code from us.
7. The function checks `consent`, then calls `read_upload` (size limit), `read_exif_gps` (reads hidden GPS from the **raw** upload) and `sanitize_image` (validates, shrinks and re-saves as JPEG, which strips that hidden data). The order matters: GPS must be read before it is stripped.
8. It picks the location source: browser GPS or map pin if the user gave one, otherwise the GPS from the photo, otherwise none.
9. It creates a `Sighting` row with `status="processing"`, saves the JPEG through `storage.put`, writes an audit line, and commits.
10. `background.add_task(jobs.process_sighting, s.id)` schedules the slow AI work for **after** the reply, so the user is not left waiting.
11. The function returns `sighting_out(s)`, a trimmed view with no embedding and no exact location. The browser gets `201 Created`.

Part C: the background job (`services/jobs.py`, `process_sighting`)

12. Loads the saved photo and calls `pipeline.process`, which finds the face and makes the 512 numbers, runs the cleanup filter, checks the cleaned face still looks like the original, and returns everything together.
13. Saves `restored.jpg` if the cleaned copy was kept, and fills in the embedding on the row.
14. If there was a GPS location, `geo.reverse_geocode` turns it into an address and a friendly area name such as "Banjara Hills, Hyderabad".
15. Sets `status` to `active` if a face was found, or `no_face` if not.
16. For an active sighting, `matching.match_sighting` compares its embedding with every **approved** missing-person record and saves any pair above the threshold as a `Match` with status `pending`.
17. `notify.notify_moderators` sends a Telegram alert, or just logs a line if Telegram is not configured.

Meanwhile in the browser: after submitting, the app moves to `MyRecords.jsx`, which checks every 4 seconds while any item says "Processing" (a `setInterval` inside a `useEffect`), so the badge flips to "Active" by itself.

Part D: the humans

18. A moderator opens **`Admin.jsx`**, which calls `GET /api/admin/matches`. `require_moderator` in `security.py` rejects anyone whose role is not `moderator`.
19. They compare the two original photos side by side, then press *Confirm match* or *Not a match*, which posts to `/api/admin/matches/{id}/decision`.
20. On confirm, the match becomes `confirmed`, a row goes into the audit log, and an email goes to the family with **no details**, only "sign in to see".
21. The family signs in, `Matches.jsx` calls `GET /api/matches/mine`, and only now does the response include the location and the sighting photo. Before confirmation, `matches.py` deliberately leaves them out.

The test `test_full_flow_and_privacy` in `backend/tests/test_flow.py` runs exactly this journey automatically, including checks that the family could **not** see the location before step 20. Reading that test is a good second pass through this section.

React for Angular developers

React has far fewer concepts than Angular. There are no modules, no decorators, and no separate template files: **a component is just a JavaScript function that returns HTML-like markup**, and everything else is a convention built on that.

Concept map

Angular	React (this project)
`@Component` class + `.html` + `.css`	One function in a `.jsx` file that returns JSX
Template `{{ name }}`	`{name}` inside JSX
`*ngIf="x"`	`{x && <div>...</div>}` or `{x ? <A/> : <B/>}`
`*ngFor="let m of items"`	`{items.map(m => <Row key={m.id} .../>)}` (the `key` is required)
`@Input() title`	A **prop**: `function Card({ title })`
`@Output() clicked`	A prop that is a function: `onClick={handleClick}`
`(click)="save()"`	`onClick={save}`
`[(ngModel)]="name"`	`value={name} onChange={e => setName(e.target.value)}`
`[class.active]="on"`	`className={on ? "active" : ""}`
Component property that triggers re-render	**State**: `const [name, setName] = useState("")`
`ngOnInit`, `ngOnDestroy`	`useEffect(() => { ...setup; return () => ...cleanup }, [deps])`
Injectable service, `providedIn: 'root'`	A **context** (`auth.jsx`) or a plain module (`api.js`)
`async` pipe / RxJS	`async/await` with `fetch` (no RxJS in this project)
`RouterModule`, `routerLink`, guards	`react-router-dom`: `<Route>`, `<Link>`, and the `RequireAuth` wrapper in `App.jsx`

Angular	React (this project)
`ng serve` / `ng build`	`npm run dev` / `npm run build` (Vite)
`angular.json`, `environment.ts`	`vite.config.js`, `.env` with `VITE_API_URL`

JSX in 30 seconds

JSX is HTML written inside JavaScript. Three rules cover most of it:

- Curly braces `{ }` drop back into JavaScript: `<h3>{m.full_name}</h3>`.
- Use `className` instead of `class`, and `htmlFor` instead of `for`.
- A component must return **one** parent element. Wrap siblings in an empty `<> ... </>` if you have no wrapper.

State: the one idea that matters most

In Angular you change `this.name = "x"` and the template updates. In React you must call the setter, because that is how React knows to redraw:

```
const [note, setNote] = useState("");          // note = current value,
setNote = the way to change it

<textarea value={note} onChange={(e) => setNote(e.target.value)} />
```

Never write `note = "x"` directly; nothing would update. This is the single most common beginner mistake.

`useEffect`: where side effects live

The second argument (the dependency array) says when to re-run:

```
useEffect(() => { load(); }, []);            // empty: run once when the
component appears (like ngOnInit)
useEffect(() => { ... }, [id]);              // run again whenever `id`
changes
useEffect(() => {
  const t = setInterval(load, 4000);
  return () => clearInterval(t);             // the returned function is the
cleanup (like ngOnDestroy)
}, [anyProcessing]);
```

The polling in `MyRecords.jsx` is the last example. Forgetting the cleanup would leave timers running after you leave the page.

Context: React's version of a shared service

`auth.jsx` creates an `AuthProvider` that holds the signed-in user. It wraps the whole app in `main.jsx`. Any component then calls `const { user, logout } = useAuth();`. It plays the same role as injecting an `AuthService` in Angular.

Hooks are the functions that start with `use`

`useState`, `useEffect`, `useCallback`, `useNavigate`, `useAuth`. One rule: call them at the top level of a component, never inside an `if` or a loop.

Routing

`App.jsx` lists every page:

```
<Route path="/records" element={<RequireAuth><MyRecords /></RequireAuth>} />
```

`RequireAuth` is a tiny component that redirects to `/login` when nobody is signed in. It plays the role of an Angular route guard, written as a wrapper instead of a class.

What you will miss and what you will not

You will not find dependency-injection containers, modules, `NgModule` declarations, or two-way binding. In exchange, every page here is one readable file where the data, the logic and the markup sit together.

Frontend tour: every file in `frontend/`

The frontend is a **single-page app**, like an Angular app: the browser loads one HTML page once, and React swaps what is shown as the URL changes. It is built with **Vite**, which plays the role of the Angular CLI.

Root files

File	What it does
`index.html`	The one HTML page. It has an empty `<div id="root">` and loads `src/main.jsx`.
`package.json`	Dependencies (React, React Router, Leaflet for maps) and the scripts `dev` and `build`.
`vite.config.js`	Vite setup: the React plugin and port 5173.
`.env.example`	Copy to `.env`. Holds `VITE_API_URL`, the address of the backend.

`src/`: the app shell

File	What it does
`main.jsx`	The entry point, like `main.ts`. Mounts React and wraps the app in the router and `AuthProvider`.
`App.jsx`	The top navigation bar and the route table (which URL shows which page). Also contains `RequireAuth`, the sign-in guard.
`api.js`	The only place that calls `fetch`. Adds the login token, turns errors into readable messages, and stores the token in `localStorage`. It is your `HttpClient` plus interceptor in one small file.
`auth.jsx`	The shared "who is signed in" state. Exposes `useAuth()`.
`leafletIcon.js`	A small fix so map pins show their icon under Vite.
`styles.css`	All styling in one file, using CSS variables for colours.

`src/pages/` : one file per screen

File	URL	What the user does there
`Home.jsx`	`/`	Reads how it works and picks "I am looking" or "I saw someone".
`Login.jsx`	`/login`	Enters an email, then the 6-digit code, and accepts the privacy policy the first time.
`RegisterMissing.jsx`	`/missing/new`	Uploads a photo and fills in details of the missing person.
`ReportSighting.jsx`	`/sighting/new`	Uploads a photo, shares GPS or clicks the map, and adds a note.
`MyRecords.jsx`	`/records`	Sees their own records and sightings, with status badges, and can delete them or the whole account.
`Matches.jsx`	`/matches`	Sees "under review" until confirmed, then the map, address and photo. Can answer "reunited" or "not them".
`Admin.jsx`	`/admin`	Moderator desk with four tabs: possible matches, records to approve, reports, audit log.
`ReportAbuse.jsx`	`/report-abuse`	Anyone reports misuse or asks for removal. No account needed.
`Privacy.jsx`	`/privacy`	The privacy and consent text (a template to review with a lawyer).

`src/components/` : reusable pieces

File	What it does
`SecureImage.jsx`	Shows a private photo. It fetches the bytes with the login token, turns them into a temporary browser URL, and shows them. A plain `<img src=...>` cannot send a token, which is why this exists.

File	What it does
`PhotoPair.jsx`	Shows the original photo with the cleaned copy beside it and a warning not to trust the cleaned one blindly.
`PhotoField.jsx`	The file picker with a preview.
`Maps.jsx`	`LocationPicker` (click to drop a pin) and `MapView` (read-only pin), using Leaflet and OpenStreetMap.
`ui.jsx`	Small shared bits: status badges, error box, date formatter, help text for each status.

How the pieces talk

- Every page calls `api.get` or `api.post` from `api.js`, never `fetch` directly.
- `api.js` reads the token that `auth.jsx` saved at login.
- If any call returns 401 (token expired), `api.js` clears the token and fires an event; `auth.jsx` listens and signs the user out, and `RequireAuth` sends them to `/login`.
- Pages show backend errors by putting the message text in `<ErrorBox>`.

Run it on your computer

You need **Python 3.10 or newer** and **Node 18 or newer**. Check with `python --version` and `node --version` (on some systems the command is `python3`). Unzip `reunite.zip`, then open two terminals.

Terminal 1: the backend

```
cd reunite/backend
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # Windows: copy .env.example .env
```

Open `backend/.env` in any editor and change these two lines:

```
FACE_BACKEND=mock
MODERATOR_EMAILS=your-email@example.com
```

`mock` means "fake face matching, good enough to click through the screens". Use your own email in `MODERATOR_EMAILS`: signing in with that address gives you the review desk.

```
uvicorn app.main:app --reload
```

You should see `Uvicorn running on http://127.0.0.1:8000`. Open `http://localhost:8000/docs` for an automatic page listing every URL, where you can try them without the React app (like Swagger).

Terminal 2: the frontend

```
cd reunite/frontend
cp .env.example .env
npm install
npm run dev
```

Open `http://localhost:5173`.

Try the whole journey (about 5 minutes)

Use one normal window and one private/incognito window so you can be two people.

1. **Window A, as the family:** sign in with `family@test.com`. With no email server set up, the 6-digit code is shown on the screen. Register a missing person with any photo of a face.
2. **Window B, as you the moderator:** sign in with the email you put in `MODERATOR_EMAILS`. Click **Review**, then **Records to approve**, and approve it.
3. **Window A:** sign out. Sign in as `finder@test.com`. Choose *I saw someone*, upload **the same photo**, and allow or skip location.
4. **Window B:** open **Review**, **Possible matches**. You should see a pair with a very high score. Press **Confirm match**.
5. Sign in again as `family@test.com` and open **Matches**. Now you see the location, map and photo.

If you upload two different photos in step 3, there should be no match. That is the system working.

Turning on real face matching later

```
pip install -r requirements-ml.txt
```

Then set `FACE_BACKEND=insightface` in `.env` and restart. The first photo is slow because the model (a few hundred MB) downloads once. Use different photos of the same person in different lighting to see real behaviour, and remember that it is imperfect, which is why moderators exist.

Run the automatic tests

```
cd backend
pip install pytest
python -m pytest -q
```

You should see `8 passed`. Whenever you change backend code, run this again. It is your safety net.

Common errors and what they mean

What you see	Why	Fix
`ModuleNotFoundError: No module named 'fastapi'`	The virtual environment is not active, or packages are not installed	Run `source .venv/bin/activate`, then `pip install -r requirements.txt`
`command not found: python`	Your system calls it `python3`	Use `python3` and `pip3`
Browser console says "blocked by CORS"	The frontend address is not allowed by the backend	`FRONTEND_ORIGINS` in `backend/.env` must contain `http://localhost:5173`
Page says "Cannot reach the server"	Backend is not running or `VITE_API_URL` is wrong	Check Terminal 1 and `frontend/.env`
"Please wait a minute before requesting another code"	Built-in limit of one code per minute per email	Wait, or use another test email
Photo stays on "Processing…"	The first real-model run is downloading, or the job failed	Wait, check Terminal 1 for a red error line. After a restart, stuck items are marked failed so you can delete and retry

What you see	Why	Fix
"No face found"	The detector could not find a clear face	Use a sharper, front-facing photo
Sign-in code not on screen	`ENV=prod` or an email server is configured	Set `ENV=dev` and leave `SMTP_HOST` empty while learning
You edited `.env` and nothing changed	The backend reads `.env` only at start	Stop and restart `uvicorn`

One more habit worth building: when something fails, read the **last line** of the Python error first. It names the exact problem, and the lines above it show which file and line caused it.

Learn by changing things

Reading only goes so far. These exercises start tiny and each one touches a new part of the stack. Do them in order, and run `python -m pytest -q` after backend changes.

Exercises

- [ ] **1. Change a word (React).** In `frontend/src/pages/Home.jsx`, change the heading. Save, and watch the browser update by itself.
- [ ] **2. Change a setting (Python).** In `backend/.env`, set `MAX_UPLOAD_MB=2`, restart, and upload a large photo. Read the error message in the browser, then find where it comes from in `services/images.py`, function `read_upload`.
- [ ] **3. Change a limit.** In `routers/sightings.py`, change `10, 3600` to `2, 3600`. Upload three sightings and watch the third get blocked. Change it back.
- [ ] **4. Add a field (full stack).** Add `"Hair colour"` to a missing-person record. Touch four places: a column in `models.py` (`MissingPerson`), a `Form(...)` parameter in `routers/missing.py`, the `MissingOut` class and `missing_out` function in `schemas.py`, and an input in `RegisterMissing.jsx`. Delete `backend/data/app.db` first, because the app does not migrate old tables yet.
- [ ] **5. Add a new endpoint.** In `routers/admin.py`, copy `stats`, rename it, and return something new, such as the number of users. Open `/docs`, find it, and click *Try it out*.
- [ ] **6. Add a new page.** Copy `ReportAbuse.jsx` to a new page, add a `<Route>` in `App.jsx` and a `<NavLink>`. You have now touched routing, components and state.

- [ ] **7. Write a test.** In `tests/test_flow.py`, copy `test_access_control` and assert that a signed-in plain user gets `403` on `/api/admin/audit`.
- [ ] **8. Read the log.** Start the backend and upload a sighting. In Terminal 1, find the lines for the request and the background job, and match each to a step in the walkthrough above.

Where to look when you want to change X

I want to change...	Open this
How strict face matching is	`MATCH_THRESHOLD` in `.env` (lower finds more candidates, and more wrong ones)
Which cleanup mode is used	`RESTORE_BACKEND` in `.env`
Maximum photo size or upload count	`MAX_UPLOAD_MB` in `.env`, and the numbers in `user_rate_limit(...)` in routers
What the family sees before a match is confirmed	`routers/matches.py`
The email text	`notify.py` and `_email_family` in `routers/admin.py`
Colours and layout	`frontend/src/styles.css`, the variables at the top
The privacy wording	`frontend/src/pages/Privacy.jsx`
How a face becomes numbers	`services/faces.py`

Glossary

Term	Plain meaning
API	The set of URLs the backend offers. The frontend calls them and gets JSON back.
Endpoint	One URL plus method, such as `POST /api/sightings`.
JSON	Text format for data, like `{ "name": "Asha" }`.

Term	Plain meaning
FastAPI	The Python web framework used for the backend.
Uvicorn	The program that actually runs a FastAPI app, like `node server.js`.
pip / venv	Python's package installer and its private package folder.
ORM, SQLAlchemy	A library that lets you use Python classes instead of writing SQL.
SQLite	A database stored in one local file. Zero setup, used for learning.
Postgres, pgvector	A real database server, plus an add-on that can search by similarity of number lists.
Embedding	The list of 512 numbers that represents a face. Similar faces give similar lists.
Cosine similarity	A score from -1 to 1 for how alike two embeddings are. Higher means more alike.
Threshold	The score above which a pair is suggested to a moderator.
OTP	One-time password: the 6-digit code emailed to sign in.
JWT	The signed login token the browser sends with each request.
CORS	A browser rule: a site may call another address only if that address says it is allowed.
EXIF	Hidden metadata inside a photo, such as GPS and camera model.
Reverse geocoding	Turning latitude and longitude into an address.
Background task	Work that runs after the reply has been sent.
Rate limit	A cap on how often someone may call an endpoint.
JSX	HTML-like syntax written inside JavaScript.

Term	Plain meaning
Hook	A React function starting with `use`, such as `useState`.
Prop / state	Data passed into a component / data a component owns and can change.
Context	Data shared with every component below it, like an injected service.
Vite	The tool that runs and builds the React app, like the Angular CLI.

Good next steps

Once you are comfortable here, the next topics worth learning are database migrations with Alembic (so adding a column does not need a database reset), a proper job queue for background work, and deploying with the steps in the project `README.md`. Before any real-world use, read the "Read this before real use" section of that README: it covers face-model licences, accuracy limits, legal review, and training for moderators.